previous next start top

Implementing, and Understanding Type Classes

- Introduction
- Dictionary passing
- Type classes as macros
- Intensional type analysis
- The pioneering work of Stefan Kaes
- Brief bibliography

- Canonical structures: (type-class) overloading resolution as logic programming
- Stretching Type Classes: Unusual applications and arguments about type classes in Haskell

Introduction

Computational abstractions -- higher-order functions, continuations, modules, processes, automatic memory management -- have made programs faster to write, easier to show correct and to reuse. And yet there is, often subconscious, resistance to abstractions: they appear ritualistic, formal -- too abstract. One may feel getting lost. To overcome the mistrust for an abstraction it may help to look at its realization, to see what is being abstracted away. The awareness of low-level implementation details brings the appreciation of an abstraction and the intuitive explanation for it.

This page looks behind the scenes of the abstraction of *parametric overloading*, also known as bounded polymorphism, or just `type classes'. Seeing the implementation makes type classes appear simpler, friendlier, more comfortable to use. Type-class declarations and type-class constraints are no longer incantations to memorize: they suddenly make sense. Knowing the tedious job a Haskell compiler (GHC) is doing for us helps us appreciate more the convenience of type classes.

Dictionary passing, although best known, is not the only implementation of type classes. Historically first were static specialization and run-time resolution (intensional type analysis), both introduced in the pioneering work by Stefan Kaes, the father of parametric overloading. He presented the type system and proved its soundness, described the type inference algorithm, and proved the soundness and consistency of the two implementations of what is now known as type classes. It is shameful that his name is almost forgotten; his strategies are still in wide use however. Local type classes and instances introduced in his paper still await recognition.

Dictionary passing makes it easier, in retrospect, to understand the other two implementation strategies. Therefore, we describe it first and in detail. We explain by example, juxtaposing Haskell code with the corresponding code in a language with no type classes (OCaml). The implementation language could be any other higher-order language, including GHC Core. For the sake of explanation, we restrict ourselves to single-parameter, non-constructor type classes, such as `Num`, `Eq`, `Show`. The other two implementation strategies are presented next, illustrating the algorithms from the Kaes' paper, in modern terms. We conclude with the brief bibliography on implementing and understanding type classes. Finally, we point out the insightful further development: user-programmable overloading inference, aka canonical structures.

The first version of this article has been posted on the Caml mailing list on Thu Mar 8 22:36:58 2007.

Dictionary passing

The most well-known technique of implementing type classes is so-called dictionary passing. Although historically the second, it is the first to study as it helps understand the others. We explain the dictionary passing by a progression of examples that cover the most common patterns of Haskell98-like type classes, instances and polymorphic functions: from simple `Show` to polymorphic recursion. The examples are inspired by the numeric hierarchy of Haskell. They contrast Haskell98 code with its `translation' into OCaml, a sample higher-order language without type classes. Our simple subset of OCaml may be regarded as a friendlier dialect of GHC Core, an intermediate type-class--free language of GHC. This translation exposes the compilation strategy, explaining what happens with type classes as GHC translates the source code to Core. We can use the translation to bring type classes to any functional language -- although the lack of the syntactic sugar bestowed by the type class abstraction is jarring.

We start with the simplest overloaded function to show values of various types as a string: the `Show` type class. (The Haskell standard Prelude class `Show` is a bit more complex and optimized than ours; the main idea is the same.) Below is the declaration of our simple `Show` and two of its instances -- and the corresponding OCaml code. We always write type signatures, even though they are inferred, except for the very last example of polymorphic recursion.

Haskell

```
class Show a where
  show :: a -> String

instance Show Bool where
  show True  = "True"
  show False = "False"

instance Show Int where
  show x = Prelude.show x  -- internal
```

OCaml

```
type 'a show = {show: 'a -> string}

let show_bool : bool show =
  {show = function
    | true  -> "True"
    | false -> "False"}

let show_int : int show =
  {show = string_of_int}
```

The type-class declaration `Show a` in Haskell translates to the data type declaration for the record `'a show`: the dictionary. The type-class method name becomes the label in the dictionary.

As an example of using the just introduced `show`, we define an overloaded function to print values of various types, and instantiate it to print a boolean.

Haskell

```
print :: Show a => a -> IO ()
print x = putStrLn $ show x
```

```
test_print :: IO ()
test_print = print True
```

OCaml

```
let print : 'a show -> 'a -> unit =
  fun {show=show} x -> print_endline (show x)

let test_print : unit =
  print show_bool true
```

Let's compare the inferred types of the `print` function, in Haskell and OCaml. They are, respectively:

```
print :: Show a =>  a -> IO ()
print : 'a show -> 'a -> unit
```

Both in Haskell and OCaml, `print` is a polymorphic function. However, its polymorphism is restricted, or bounded: `print` applies to values of only those types that are showable, that is, are members of the `Show` class -- and we have the evidence for their membership. This is (painfully) explicit in OCaml's `print`: the evidence of being showable, the dictionary, is the explicit argument. It is the programmer's responsibility to find this evidence and pass it to `print`: see `test_print`'s code. In Haskell, the polymorphism restriction takes the form of a so-called type-class constraint `Show a`. It, too, may be regarded as the argument carrying the membership evidence. However, it is the Haskell compiler that finds and passes the evidence. The `Show a` argument is therefore implicit, which is indicated by the double-arrow in `print`'s type.

The OCaml type of `print` reveals the nature of bounded polymorphism. An unbounded polymorphic function such as `id : 'a -> 'a` corresponds to the universally quantified proposition `forall a. a -> a`. The function `print` witnesses the proposition `a -> unit` quantified only over a part of the domain of discourse. The predicate `show` decides the membership in that part: we assert `a -> unit` only when `show(a)`. The proposition thus reads `forall a. show(a) -> (a -> unit)` -- which is the type of `print` modulo stylistic differences.

Next is the (simplified) `Num` type class, whose methods have a different pattern of overloading: the method `fromInt` is overloaded on the result type, and the method `(+)` is binary.

Haskell

```
class Num a where
  fromInt :: Int -> a
  (+)     :: a -> a -> a

sum :: Num a => [a] -> a  -- sample function
sum ls = foldr (+) (fromInt 0) ls
```

OCaml

```
type 'a num =
    {fromInt: int -> 'a;
     add:     'a -> 'a -> 'a}

let sum : 'a num -> 'a list -> 'a =
  fun {fromInt = fromInt; add = add} ->
  fun ls -> List.fold_right add ls (fromInt 0)
```

The `Num` type class has two methods; therefore, the corresponding dictionary record `num` has two fields. The instances for `Bool` and `Int`, and their translations, are straightforward. We show only the `Int` instance:

Haskell

```
instance Num Int where
  fromInt x = x
  (+)       = (Prelude.+)
```

OCaml

```
let num_int : int num =
  {fromInt = (fun x -> x);
   add     = Stdlib.(+)}
```

The polymorphic function `print_incr`, below, to print an incremented value, depends on two constraints:

Haskell

```
print_incr :: (Show a, Num a) => a -> IO ()
print_incr x = print $ x + fromInt 1

print_incr_int :: Int -> IO () -- A sample instantiation
print_incr_int x = print_incr x
```

OCaml

```
let print_incr : ('a show * 'a num) -> 'a -> unit =
  fun (show_dict, {fromInt=fromInt;add=(+)}) ->
  fun x -> print show_dict (x + fromInt 1)

let print_incr_int : int -> unit = fun x ->
  print_incr (show_int,num_int) x
```

The inferred type of Haskell's `print_incr` has the pair of constraints `(Show a,Num a)`. In the translation, it becomes the pair of dictionaries, `'a show` and `'a num`, passed to `print_incr` as the explicit argument. OCaml's `print_incr` needs `show_dict` to pass it to the parametrically overloaded `print` (talked about earlier). The numeric operations extracted from `num_dict` are used when incrementing. All this boilerplate of dictionary passing and extraction is explicit in OCaml, but implicit in Haskell. A Haskell compiler handles this boilerplate for us.

To instantiate a bound-polymorphic function in Haskell we merely have to use it in a specific type context or give a specific type, see Haskell's `print_incr_int`. The type checker will verify that the specific type, `Int` in our case, is the member of `Show` and `Num`. These constraints of `print_incr` become resolved and no longer appear in the type of `print_incr_int`. On the OCaml side, we don't just make the type variable `'a` to be `int` and let the type checker verify the constraint satisfaction. It is the programmer who has to prove that the constraints are indeed satisfied: the programmer has to find and explicitly pass the dictionaries `show_int` and `num_int`, as the proof that `int` is indeed a member of `Show` and `Num`. The type class abstraction does such proofs for us, searching for dictionaries and combining them in the complete evidence to pass to a parametrically overloaded function.

The next common pattern is an instance with a constraint: a `Show` instance for all list types `[a]` where the element type `a` is also restricted to be a member of `Show`.

Haskell

```
instance Show a => Show [a] where
  show xs = "[" ++ go True xs
   where
     go _ []    = "]"
     go first (h:t) = (if first then "" else ", ") ++ show h ++ go False t

testls :: String
testls = show [1::Int,2,3]
```

OCaml

```
let show_list : 'a show -> 'a list show = fun {show=show} ->
  {show = fun xs ->
   let rec go first = function
```

```
      | []   -> "]"
      | h::t -> (if first then "" else ", ") ^ show h ^ go false t
   in "[" ^ go true xs}

let testls : string =
  (show_list show_int).show [1;2;3]
```

The `instance Show a => Show [a]` now translates to a function, which receives the `'a show` dictionary, the evidence that `'a` is a member of `Show`, and produces the evidence that `'a list` is also a member. As before `` `=>' `` becomes `` `->' `` in the translation. The occurrence `show h` in the Haskell code is not a recursive reference to the list show being defined. Rather, it refers to the `show` at a different type, the type of list elements. The OCaml code makes this reference clear. The specialization, `testls`, again involves more work on the OCaml side: we have to build the proof that `int list` is showable, by finding the evidence that `int` is showable and passing it to `show_list` to obtain the desired proof, that is, the function for showing integer lists.

For the final examples we need a class of comparable types:

```
class Eq a where
  (==) :: a -> a -> Bool
```

Its `Bool` and `Int` instances, and the corresponding dictionary `type 'a eq = {eq: 'a -> 'a -> bool}` are straightforward and elided. More interesting is the type class with a super-class and a default method:

Haskell

```
class (Eq a, Num a) => Mul a where
  (*) :: a -> a -> a
  x * _ | x == fromInt 0 = fromInt 0
  x * y | x == fromInt 1 = y
  x * y = y + (x + (fromInt (-1))) * y

instance Mul Bool where
  -- default

instance Mul Int where
  x * y = (Prelude.*) x y  -- internal
```

OCaml

```
type 'a mul = {mul_super: 'a eq * 'a num;
               mul: 'a -> 'a -> 'a}

let mul_default : 'a eq * 'a num -> 'a mul =
  fun (({eq=eq},{fromInt=fromInt;add=(+)}) as super) ->
  {mul_super = super;
   mul = let rec loop x y = match () with
  | () when eq x (fromInt 0) -> fromInt 0
  | () when eq x (fromInt 1) -> y
  | () -> y + loop (x + (fromInt (-1))) y
  in loop}

let mul_bool : bool mul = mul_default (eq_bool,num_bool)

let mul_int : int mul =
  {mul_super=(eq_int,num_int); mul=Stdlib.( * )}
```

The default code for the multiplication recursively refers to the multiplication being defined. This reference happens at the same type: the recursion is ordinary, not polymorphic. Again this is apparent in the translation.

The constraint `(Eq a, Num a)` in the class `Mul` declaration has a subtlety, revealed in the OCaml translation. Recall the earlier `instance Show a => Show [a]`. One may read it as a conditional (qualified)

declaration: the type `[a]` is a member of the `Show` class provided `a` is a member. The OCaml translation made the qualification clear: `show_list` is a function that takes an `'a show` dictionary (the evidence of the `Show a` membership) and returns `'a list show`: the evidence that the list type is also showable. One may be tempted to regard `class (Eq a, Num a) => Mul a` similarly, as a qualified declaration. It is not -- and the OCaml translation makes it clear. We see that `mul_bool` and `mul_int` are not functions: they are dictionaries, which include the pair of dictionaries `'a eq` and `'a num`. The two are hence *provided* by the `'a mul` dictionary.

The fact that `Mul a` provides, rather than requires, the `Num a` and `Eq a` membership evidence is clear from the type of a sample function: computing dot-product.

Haskell

```
dot :: Mul a => [a] -> [a] -> a
dot xs ys = sum $ zipWith (*) xs ys

test_dot :: Int
test_dot = dot [1,2,3] [4,5,6]
```

OCaml

```
let dot : 'a mul -> 'a list -> 'a list -> 'a =
  fun {mul_super=(eq,num);mul=mul} ->
  fun xs ys -> sum num @@ List.map2 mul xs ys

let test_dot : int =
  dot mul_int [1;2;3] [4;5;6]
```

The OCaml translation `dot` receives only `'a mul` but does not only multiplication but also addition. One may feel that the constraint in the class declaration should have been written as `class (Eq a, Num a) <= Mul a`. In fact, such a syntax has been suggested. The different interpretation of constraints in instance and class declarations is known, but not well, and can be confusing.

The final example deals with polymorphic recursion. Type signatures become mandatory.

Haskell

```
print_nested :: Show a => Int -> a -> IO ()
print_nested 0 x = print x
print_nested n x = print_nested (n-1) (replicate n x)

test_nested = do
  n <- getLine
  print_nested (read n) (5::Int)
```

OCaml

```
let rec print_nested : 'a. 'a show -> int -> 'a -> unit =
  fun show_dict -> function
    | 0 -> fun x -> print show_dict x
    | n -> fun x -> print_nested (show_list show_dict) (n-1) (replicate n x)

let test_nested =
  let n = read_int () in
  print_nested show_int n 5
```

At first blush, the code is straightforward. After seeing the output one realizes that the type of the printed value, the deeply nested list `[[...[Int]...]]`, is not statically known. It depends on the value of `n` received from the user at run-time. Since we do not know the exact type of `x` at compile time, we cannot statically build the evidence that it is showable. The compiler must arrange for building such evidence

dynamically. The OCaml code illustrates such an arrangement: as we add one more `list` to the type, we transform the current `show_dict` with one more `show_list`.

The explicit construction, deconstruction and passing of dictionaries in the OCaml code is annoying. What makes type classes popular in Haskell is the hiding of all this plumbing. The convenience increases when two type classes are involved, e.g., `(Show a, Num a)` in `print_incr`. In Haskell `(Show a, Num a)` and `(Num a, Show a)` are the same constraints -- but the corresponding types in OCaml `('a show * 'a num)` and `('a num * 'a show)` are different. Actually, OCaml has extensible records, in which the order of fields does not matter. These records are more appropriate for modeling dictionaries.

In conclusion, we have described the dictionary passing implementation of type classes by the way of a translation to OCaml, a sample higher-order language. The double-arrow is translated to the ordinary arrow: the type class constraint becomes the explicit dictionary argument, the evidence of the constraint satisfaction. Therefore, in OCaml we have to explicitly pass the dictionary argument to all bounded polymorphic functions. In Haskell, the dictionary is an implicit argument and the Haskell compiler does a great job of filling it in where needed, hiding the argument from the user. Overloading over type constructor (e.g., `Monad` class) is conceptually similar, but requires type constructor polymorphism in the language. Haskell constructor classes hence need OCaml functors. Conversely, OCaml and SML modules (including sealing, generative and applicative functors and recursive structures) can be emulated as Haskell constructor type classes, see the bibliography at the end.

We now look at the other two implementations of type classes and contrast them with the dictionary passing implementation, using the examples from the present section.

References

typeclass_code.hs [3K]
typeclass_code.ml [5K]
The complete code for the series of examples, in Haskell (using type classes) and OCaml (implementing type classes with dictionary passing)

final_dic.ml [3K]
Using extensible records for overloading

Type classes as macros

Historically the first implementation strategies for parametric overloading were static monomorphization and dynamic intensional type analysis. They were introduced in the Kaes 1988 paper, in a (overly) concise and formal way with no examples and in a slightly foreign, by now, terminology. We present these techniques in modern terms and illustrate on concrete examples, relating them to dictionary passing, compilation of unbounded polymorphism, and partial evaluation. We will see the similarity with C++ templates and generic functions (both of which appeared after Kaes work). Static monomorphization is presented here, as an elaboration of the algorithm from Section 4.2 of Kaes' 1988 paper. The intensional type analysis is in the next section.

Monomorphization takes the type-checked code with type classes and re-writes it into the code with no type classes and no bounded polymorphism. All overloading is resolved. It is a type directed program transformation; it assumes that all identifiers in the source program are annotated with their types -- which they will be after the type inference and type checking passes of the compiler. We explain the monomorphization on the `Show` example from the previous section. It is repeated below, with type annotations on relevant identifiers. This is the input to monomorphization.

```
class Show a where
  show :: a -> String
```

```
instance Show Bool where
  show::Bool->String = bool_to_string

instance Show Int where
  show::Int->String = int_to_string

instance Show a => Show [a] where
  show::([a]->String) xs =
    strings_to_string $ map (show::a->String) xs

test_show :: String
test_show = (show::Bool->String) True

print :: forall a. Show a => a -> IO ()
print x = putStrLn $ (show::a->String) x

print_ints :: IO ()
print_ints = (print::[Int]->IO ()) [(1::Int),2,3]
```

To avoid clutter, we have assumed the primitives `bool_to_string :: Bool -> String`, `int_to_string :: Int -> String` and `strings_to_string :: [String] -> String`.

Monomorphization takes the source program and the resolution environment, initially empty. The resolution environment

```
type REnv = Map TypedId Exp
type TypedId = (Id,Typ)
```

is essentially a set of definitions. `REnv` relates a globally or locally defined identifier, along with its type, to a source expression: the right-hand-side of its definition. Monomorphization works definition-by-definition, expression-by-expression, re-writing source expressions into the target expression and possibly updating the `REnv`. It recursively replaces all overloaded identifiers with whatever they resolve to in the `REnv`, until no overloading is left. Let us illustrate monomorphization by tracing it through the example source program above.

The first in the source program is the class declaration for `Show`. It is re-written to nothing and `REnv` remains empty. The class declaration is used only for type checking and can be erased afterwards. Next comes `instance Show Bool`. We also output nothing, but now extend `REnv` with the mapping of `(show,Bool->String)` to `bool_to_string`. After processing the other instances of `Show`, the output program is empty but the `REnv` contains three mappings:

```
(show,Bool->String) --> bool_to_string
(show,Int->String)  --> int_to_string
(show,[a]->String)  --> \xs -> strings_to_string $ map (show::a->String) xs
```

Parametrically overloaded definitions (including type class methods) thus become mappings in `REnv`. Next we see `test_show`, which is not parametrically overloaded: its type has no constraints. We re-write this definition, scanning through its body looking for the identifiers mentioned in `REnv`, to be replaced. The first seen identifier `show::Bool->String` occurs in the `REnv` with the same `Bool->String` type. It is replaced with its mapping, `bool_to_string`, and the replacement is re-scanned. The result of the re-writing is:

```
test_show :: String
test_show = bool_to_string True
```

The next definition in the source program is of `print`. Its type has a constraint, `Show`. Therefore, we output nothing and add the mapping to the `REnv`:

```
(print,a->IO ()) --> \x -> putStrLn $ (show::a->String) x
```

The final definition is `print_ints :: IO ()`. It is not overloaded, so we re-write its body. As we scan it we encounter `print::[Int]->IO ()`. The environment `REnv` has `print`, but with a more general type `a->IO()`. We hence replace `print::[Int]->IO ()` with whatever `print` is mapped in the environment, substituting `[Int]` for `a`. The result is `\x -> putStrLn $ (show::[Int]->String) x`, which is to be recursively re-written. We encounter `show::[Int]->String`, which is defined in `REnv` with a more general type, and repeat the substitution and rewriting, etc. The final, re-written program has two definitions:

```
test_show :: String
test_show = bool_to_string True

print_ints :: IO ()
print_ints =
  (\x -> putStrLn $
          (\xs -> strings_to_string $ map int_to_string xs) x)
  [(1::Int),2,3]
```

There are no type classes, no instances, no constraints, no overloaded identifiers. Overloading has been resolved and the bounded polymorphism eliminated -- monomorphized. Blindly replacing identifiers by expressions tends to bloat the code. Therefore, in practice one introduces auxiliary definitions for the specialized `show` and `print`.

The monomorphization process looks a lot like macro expansion. In fact, Kaes wrote: ``This corresponds to the view, that overloaded function definitions are not functions in the usual sense, but macros which are expanded according to an implicit type parameter.'' The similarity with C++ template instantiation is also clear. We stress however a critical difference of monomorphization from macro expansion or template instantiation: monomorphization rewrites the code *after* the type checking. The result of monomorphization shall always be well-typed, because each overloaded identifier is replaced with the expression of the same type. In contrast, C++ template instantiation may produce ill-typed code, and gigabyte-long error messages.

We have illustrated monomorphization that was formally presented in the Kaes 1988 paper. There is a different way to the same result. Let's look at the result of the dictionary passing translation of our source program (in OCaml syntax), quoted from from the previous section.

```
type 'a show = {show: 'a -> string} (* Translated class declaration *)

                                    (* Translated instances *)
let show_bool : bool show = {show=bool_to_string}
let show_int  : int show = {show=int_to_string}
let show_list : 'a show -> 'a list show =
 fun{show=show} -> {show=fun xs ->strings_to_string (List.map show xs)}

let test_show : string = show_bool.show true

let print : 'a show -> 'a -> unit =  (* Translated overloaded function *)
  fun {show=show} x -> print_endline (show x)

let print_ints : unit =              (* and its instantiation *)
 print (show_list show_int) [1;2;3]
```

Let us inline the dictionaries and the bound-polymorphic functions and do the standard constant propagation, performing dictionary function applications and record field references. The result is

```
let test_show : string =  bool_to_string true

let print_ints : unit =
  (fun x ->
    print_endline ((fun xs -> strings_to_string (List.map int_to_string xs)) x))
  [1;2;3]
```

which is exactly the output of monomorphization (but written in OCaml syntax). Thus monomorphization is a partial evaluation of the result of the dictionary-passing translation.

Monomorphization for compiling parametric overloading is similar to compiling polymorphic functions by specializing them to the types they are used at within the program. MLton, for instance, compiles polymorphic functions in this way. Both techniques treat polymorphic functions, fully or bound polymorphic, as second class, as macros. Both techniques share the same advantages and disadvantages. Since they specialize away polymorphic functions, there is no longer a need for the uniform data representation. Unboxed integers and floats greatly improve performance. The specialization enables further optimizations. Thus both monomorphization techniques compile polymorphism with no run-time overhead. On the downside, monomorphization is a whole program transformation, incompatible with separate compilation. Furthermore, monomorphization cannot be done for polymorphically recursive or higher-rank (first-class polymorphic) functions.

We have described monomorphization, the way of compiling type classes by transforming the source program into the type-class-free and overloading-free program. We have thus explained the algorithm from Section 4.2 of Kaes 1988 paper and illustrated it on the concrete example. Monomorphization turns out related to the dictionary passing translation and partial evaluation: monomorphization is the dictionary-passing translation followed by specialization of all bound-polymorphic functions and the inlining of dictionaries. In Haskell, monomorphization alone is not sufficient to compile all programs with type classes. It can be used however on the case-by-case basis; GHC does exactly that. GHC first compiles away type classes with the dictionary-passing translation, and then does partial evaluation where it is obvious and feasible, trying to fully eliminate the passing of dictionaries at run-time. It is the monomorphization that is responsible for the efficiency of programs with the State or IO monads, for example.

Intensional type analysis

Intensional type analysis is the other method of compiling type classes, complementary to monomorphization. Whereas the latter resolves all overloading at compile time, with the intensional type analysis the appropriate overloading operation is chosen at run-time. On the other hand, the intensional type analysis is modular, and compatible with separate compilation and first-class polymorphism. Whereas monomorphization evokes partial evaluation or C++ template instantiation, the intensional type analysis reminds one of generic functions or run-time type identification. The intensional type analysis for type classes was also introduced in the Kaes 1988 paper, in Section 4.3, in a terse formal way with no examples. This section illustrates and elaborates a simpler version of Kaes' algorithm, and relates it with dictionary passing.

The intensional type analysis for type classes, like monomorphization, is a program transformation that eliminates all mentioning of type classes. Its input is again the program after type inference and type checking, when all identifiers are annotated with their types. It is quite simpler than monomorphization. We explain the intensional type analysis on the example from the previous section. First we discuss the technique presented in the Kaes 1988 paper; we then improve it, as Kaes himself suggested at the end of his paper.

We explain the intensional type analysis by presenting the source -- type-annotated Haskell program -- side-by-side with its translation into OCaml. As before, OCaml is a stand-in for a higher-order (intermediate) language, including GHC Core. Just for a moment we assume the existence of primitives `is_bool: 'a -> bool option`, `is_int: 'a -> int option` and `is_list: 'a -> 'a list option`. They are *non-parametrically* polymorphic functions that examine the run-time representation of their argument and, when of the right type, cast it to a boolean, integer, etc. Later on we will do without them.

Haskell

```
class Show a where
  show :: a -> String

instance Show Bool where
  show::Bool->String = bool_to_string

instance Show Int where
  show::Int->String = int_to_string

instance Show a => Show [a] where
  show::([a]->String) xs =
    strings_to_string $ map (show::a->String) xs
```

OCaml

```
let show : 'a -> string = fun x ->
  failwith "failed overloading resolution"

let show : 'a -> string = fun x ->
  match is_bool x with
  | Some x -> bool_to_string x
  | _      -> show x

let show : 'a -> string = fun x ->
  match is_int x with
  | Some x -> int_to_string x
  | _      -> show x

let show : 'a -> string = fun x ->
  match is_list x with
  | Some x -> strings_to_string (List.map show x)
  | _      -> show x
```

Each type class instance, `Show` in our example, is translated to a definition that extends the previous definition of the method, `show`, by adding a new dispatch alternative. We essentially turn a collection of methods from type-class instances into one large `switch` statement, which dispatches on the type of the argument determined from its run-time representation. The translation of the rest of the source program is essentially the identity:

Haskell

```
test_show :: String
test_show = (show::Bool->String) True

print :: forall a. Show a => a -> IO ()
print x = putStrLn $ (show::a->String) x

print_ints :: IO ()
print_ints = (print::[Int]->IO ()) [(1::Int),2,3]
```

OCaml

```
let test_show : string =
      show true

let print : 'a -> unit =
  fun x -> print_endline (show x)

let print_ints : unit =
  print [1;2;3]
```

When the hard work of overloading resolution is shifted to run-time, it is no wonder the compilation of type class programs is trivial: erasing type class constraints and leaving the code as it was. A bounded polymorphic function becomes ordinary polymorphic. There is a price to pay for this simplicity: the loss of parametricity, of the representation-independence abstraction and of free theorems. The function `print : 'a -> unit` may look from its type like parametrically polymorphic, but it isn't. Furthermore, it may be impossible to accurately determine the type of a value from its run-time representation. Although one can tell in OCaml a non-empty integer list from an integer by examining the run-time value -- the empty list, `false` and `0` are represented identically at run-time. The intensional type analysis above cannot dispatch on the result type. There is yet another drawback, specific to Haskell: to tell the type of a Haskell value from its run-time representation, it has to be evaluated first, at least up to the head constructor. Therefore, an overloaded function is necessarily strict. Kaes was aware of these problems, and suggested, at the end of his paper, adding an argument to overloaded functions to describe the type on which to resolve the overloading.

We now elaborate Kaes' suggestion. We define a data type, GADT, that represents a type at run-time.

```
type _ trepr =
  | Int  : int trepr
  | Bool : bool trepr
  | List : 'a trepr -> 'a list trepr
```

The new translation becomes:

Haskell

```
class Show a where
  show :: a -> String

instance Show Bool where
  show::Bool->String = bool_to_string

instance Show Int where
  show::Int->String = int_to_string

instance Show a => Show [a] where
  show::([a]->String) xs =
    strings_to_string $ map (show::a->String) xs
```

OCaml

```
let show : type a. a trepr -> a -> string = fun _ x ->
  failwith "failed overloading resolution"

let show : type a. a trepr -> a -> string = function
  | Bool  -> bool_to_string
  | trepr -> show trepr

let show : type a. a trepr -> a -> string = function
  | Int   -> int_to_string
  | trepr -> show trepr

let show : type a. a trepr -> a -> string = function
  | List trepr -> fun x -> strings_to_string (List.map (show trepr) x)
  | trepr -> show trepr
```

For the rest of the program,

Haskell

```
test_show :: String
test_show = (show::Bool->String) True

print :: forall a. Show a => a -> IO ()
print x = putStrLn $ (show::a->String) x
```

```
print_ints :: IO ()
print_ints = (print::[Int]->IO ()) [(1::Int),2,3]
```

OCaml

```
let test_show : string =
      show Bool true

let print : 'a trepr -> 'a -> unit =
  fun trepr x -> print_endline (show trepr x)

let print_ints : unit =
  print (List Int) [1;2;3]
```

Comparing the translation result with that of the dictionary passing translation shows uncanny similarity. In either case, parametrically overloaded functions take an extra argument. It is the compiler which has to provide this argument and pass it around. The two translations also differ: for example, the intensional type analysis may raise a run-time overloading resolution exception (at least in principle); no such errors may occur with dictionary passing. The explicit passing of the run-time type representation likens the intensional type analysis with generic programming libraries, especially LIGD (Lightweight implementation of generics and dynamics, Cheney and Hinze, 2002), although the idea of a type/shape descriptor is common to many other libraries.

There are at least two Haskell systems that implement type classes via the intensional type analysis: JHC and Chameleon. And so does Scala, a non-Haskell system. In Scala, the `'a trepr` argument may be declared implicit -- in which case the compiler will construct the appropriate value automatically. The programmer does not need to explicitly pass these `trepr` values around, but still has to specify them in signatures. These ideas culminate in canonical structures (see the bibliography).

We have presented the intensional type analysis, the third way of compiling type classes. Whereas monomorphization is akin to C++ template instantiation and dictionary passing is similar to `vtable`, the intensional type analysis reminds one of resolving overloading by a large `switch` statement.

The pioneering work of Stefan Kaes

Stefan Kaes is the pioneer of type classes. His ESOP 1988 paper was first to make ad hoc overloading systematic. Here's the abstract of the paper:

> The introduction of unrestricted overloading in languages with type systems based on implicit parametric polymorphism generally destroys the principal type property: namely that the type of every expression can uniformly be represented by a single type expression over some set of type variables. As a consequence, type inference in the presence of unrestricted overloading can become a NP-complete problem. In this paper we define the concept of parametric overloading as a restricted form of overloading which is easily combined with parametric polymorphism. Parametric overloading preserves the principal type property, thereby allowing the design of efficient type inference algorithms. We present sound type deduction systems, both for predefined and programmer defined overloading. Finally we state that parametric overloading can be resolved either statically, at compile time, or dynamically, during program execution.

The contributions of the barely 14-page paper are hard to overstate:

- the superb introduction to the problem of overloading and the insight of combining it with parametric polymorphism, resulting in restricted (bounded) polymorphism;
- the type system of parametric overloading and the proof of its soundness: `well-typed programs with overloading don't go wrong';
- the proof of principal types and the presentation of the type inference algorithm, based on modified unification;
- the design of user-defined parametric overloading: in modern terms, local type classes and instances; extending the previous proofs of soundness and inference from built-in to user-defined, locally overloaded functions;
- static and dynamic overloading resolution algorithms -- monomorphization and intensional type analysis -- and the proofs of their correctness and consistency;
- brief mentioning of implemented extensions: constructor classes and combining parametric overloading with subtyping;
- the presented type systems, inference algorithms and compilation methods were implemented in a functional programming language SAMPLE, which has been used in Kaes group for a year.

In modern terms, Kaes' parametrically overloaded functions are single-parameter type classes with a single method. Therefore, he identifies the name of the type class with the name of the method, the overloaded identifier. He imposes restrictions similar to Haskell'98 restrictions on type classes: no overlapping instances; the type in the instance head is either a zero-arity type constant `T` or an n-ary constant `Tn` applied to n distinct type variables. The overloading is hence resolved by looking only at the outermost type constructor. The types of overloaded functions are further restricted so that the overloading can be resolved only from the argument types. This restriction seems to come from one of the compilation strategies: dynamic resolution. The paper briefly mentions a more general strategy, which should not impose such restrictions. An `overloading assumption' used throughout the paper is, in modern terms, a set of type classes and the types of their instances. For example, the Haskell type class `Eq` and its instances

```
class Eq a where eq :: a -> a -> Bool
instance Eq Int where ...
instance Eq Bool where ...
instance Eq a => Eq [a] where...
```

in Kaes' terms take the form of the following overloading assumption:

```
{eq: <$->$->Bool, {Int, Bool, [a_(eq)]}}
```

Here, `a_(eq)` is a peculiar way to specify that the type variable `a` is restricted to be a member of the type class `Eq`. The overloading type schema `$->$->Bool` is in modern terms the method signature, with `$` being the dedicated type variable of the (implicit) class declaration. Granted, the syntax of Kaes' calculus takes some time to get used to. The indisputable advantage of Haskell is its syntax for type classes.

The two parametric overloading compilation techniques from the Kaes paper have been explained above. We have illustrated a simpler version of the techniques, for globally defined parameterically overloaded functions, such as Haskell type classes. In the Kaes paper parameterically overloaded functions have local scope. Here is an example, in an imagined Haskell-like syntax.

```
let class Show a where
     show :: a -> String
in
let f =
     let instance Show () where
          show _ = "unit 1"
     in \x -> "result " ++ show x
in
let instance Show () where
       show _ = "unit 2"
in f ()
```

The example is meant to show a subtlety brought by local instances: the coherence problem. If we infer the polymorphic type for `f :: Show a => a -> String`, then the program may plausibly return `"result unit 2"`. However, if the compiler decides to optimistically specialize `f` to the concrete type `() -> String`, `show` will be resolved to the first instance and the result will be `"result unit 1"`. The compilation strategies of the Kaes paper are so designed that the result is always `"result unit 1"`, regardless of the strategy and of possible optimistic specialization of `f`. The closure bound to `f` captures with it the overloading environment. When a polymorphic closure is instantiated and applied, it uses the local instance environment at its creation site rather than the instances in effect at the call site.

It is such a shame that the outstanding pioneering work of Kaes has been entirely forgotten.

References

Stefan Kaes: Parametric overloading in polymorphic programming languages
Proc. ESOP 1988, Springer's LNCS 300, pp. 131-144
<http://link.springer.com/chapter/10.1007%2F3-540-19027-9_9>

Stefan Kaes: Parametrischer Polymorphismus, Ueberladungen und Konversionen
Dissertation: TU Darmstadt, Fachbereich Informatik, 2005
<http://tuprints.ulb.tu-darmstadt.de/epda/000544/>
In this dissertation defended many years after the original papers, Stefan Kaes systematizes his approach, describes the relationship with Haskell type classes, and generalizes to a constraint-based inference along the lines of HM(X).
I thank Johannes Emerich for pointing out this dissertation.

Brief bibliography

Stefan Kaes. Parametric overloading in polymorphic programming languages
Proc. ESOP 1988, Springer's LNCS 300, pp. 131-144
Stefan Kaes. Type Inference in the Presence of Overloading, Subtyping and Recursive Types
Proc. 1992 ACM Conference on LISP and Functional Programming, ACM LISP Pointers, 1992, v5, N1, pp. 193-204
The two pioneering papers by Kaes.

Philip L. Wadler and Stephen Blott. How to Make Ad-Hoc Polymorphism Less Ad Hoc
Proc. POPL 1989, pp. 60-76

Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, Philip L. Wadler. Type Classes in Haskell
ACM Transactions on Programming Languages and Systems (TOPLAS), 1996, v18, N2, pp. 109-138

Derek Dreyer, Robert Harper, Manuel M. T. Chakravarty and Gabriele Keller. Modular type classes
Proc. POPL 2007, pp. 63-70

Stefan Wehr and Manuel M. T. Chakravarty. ML Modules and Haskell Type Classes: Constructive Comparison
Proc. APLAS 2008, LNCS 5356, pp. 188-204

P. J. Stuckey and M. Sulzmann. A theory of overloading
ACM Transactions on Programming Languages and Systems (TOPLAS), 27(6):1-54, 2005

Assia Mahboubi, Enrico Tassi. Canonical Structures for the working Coq user
doi:10.1007/978-3-642-39634-2_5
See also: Canonical structures for a working ML programmer, or: implicits for the masses

Translucent applicative functors in Haskell: contrasting the module system of OCaml with Haskell type classes.

Last updated May 3, 2021

This site's top page is **http://okmij.org/ftp/**
oleg-at-okmij.org
Your comments, problem reports, questions are very welcome!
Generated by MarXere